

Grafos e representação de grafos

Algoritmos e Estruturas de Dados 2

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

10 de setembro de 2026

1. O que é um grafo: definições básicas
- ↓
2. Leques, graus e contagem de arcos
- ↓
3. Grafos não-dirigidos e subgrafos
- ↓
4. Matriz de adjacências (em C)
- ↓
5. Listas de adjacência (em C)
- ↓
6. Qual representação escolher?

Material baseado em Feofiloff, *Algoritmos para Grafos (em linguagem C)*, IME-USP.

Definições básicas

Problema

Muitos problemas são sobre *objetos* e *ligações* entre eles:

- páginas da Web e os *links* entre elas;
- cidades e estradas;
- tarefas e suas dependências;
- pessoas e amizades.

Precisamos de um único modelo para todos eles.

Solução

O **grafo**: um modelo combinatório mínimo — um conjunto de *vértices* e um conjunto de *arcos*.

Resolvido o problema no grafo, a solução vale para todas as aplicações que ele modela.

Definição

Um **grafo** é um par de conjuntos: um conjunto de **vértices** e um conjunto de **arcos**. Cada **arco** é um *par ordenado* de vértices: o primeiro é a **ponta inicial** e o segundo é a **ponta final**. O arco de ponta inicial v e ponta final w é denotado por $v-w$.

Intuição e convenções deste curso

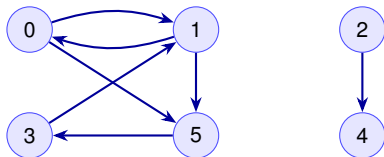
- Arcos são *dirigidos*: $v-w$ sai de v e entra em w .
- A presença de $v-w$ é independente da presença de $w-v$.
- **Não há laços**: a ponta final é sempre diferente da inicial.
- **Não há arcos paralelos**: arcos são *meros pares* de vértices, logo dois arcos distintos não podem ter as mesmas pontas.

“Grafo dirigido” (ou *digrafo*) é sinônimo; o adjetivo é redundante aqui.

Um grafo concreto

Grafo sobre os vértices 0..5 com o conjunto de arcos

0-1 0-5 1-0 1-5 2-4 3-1 5-3



0-1 e 1-0 são **antiparalelos**: a ponta inicial de um é a ponta final do outro. Note que 4 não tem nenhum arco saindo dele.

Definição

Um vértice w é **vizinho** de v (ou **adjacente** a v) se $v-w$ é um arco do grafo. O **tamanho** de um grafo com V vértices e A arcos é a soma $V + A$.

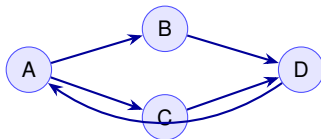
Cuidado

A relação de vizinhança **não é simétrica**: w pode ser vizinho de v sem que v seja vizinho de w .

No grafo anterior: os vizinhos de 0 são 1 e 5; o vizinho de 3 é 1; o vértice 4 não tem vizinho algum. Ali $V = 6$, $A = 7$ e o tamanho é 13.

Exemplo

A estrutura da rede WWW pode ser representada por um grafo: os **vértices** são as páginas HTML e os **arcos** são os *links* que apontam de uma página para outra. Navegar na rede é pular de um vértice a outro seguindo os arcos.



A direção importa: A aponta para B, mas B não aponta para A. É exatamente isso que a definição de arco como *par ordenado* captura.

O que aprendemos

- Grafo = conjunto de vértices + conjunto de arcos (pares ordenados).
- Notação $v-w$: sai de v , entra em w .
- w é vizinho (adjacente) de v se $v-w$ é arco; relação não simétrica.
- Sem laços e sem arcos paralelos, por convenção deste curso.
- V vértices, A arcos, tamanho $V + A$.

Leques e graus

Problema

Como medir a “importância local” de um vértice? Como saber quantos arcos um grafo pode ter, e portanto quanta memória vamos gastar para guardá-lo?

Solução

Contar os arcos que saem e que entram em cada vértice: os **graus**. Com eles obtemos limites sobre A e classificamos grafos em *densos* e *esparsos*.

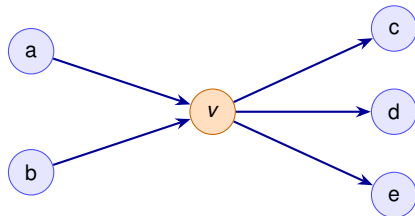
Definição

O **leque de saída** (*fan-out*) de um vértice é o conjunto de todos os arcos que saem dele; o **leque de entrada** (*fan-in*) é o conjunto dos arcos que entram nele.

O **grau de saída** (*outdegree*) é o tamanho do leque de saída; o **grau de entrada** (*indegree*) é o tamanho do leque de entrada.

Vértices com nome próprio

- **fonte** (*source*): grau de entrada nulo;
- **sorvedouro** (*sink*): grau de saída nulo;
- **isolado**: graus de entrada e de saída ambos nulos.



leque de entrada: 2 arcos

leque de saída: 3 arcos

Aqui o grau de entrada de v é 2 e o grau de saída é 3. Se o leque de entrada fosse vazio, v seria uma **fonte**; se o leque de saída fosse vazio, um **sorvedouro**.

Propriedade

A soma dos graus de *saída* de todos os vértices é igual ao número de arcos do grafo. O mesmo vale para a soma dos graus de *entrada*.

Consequência

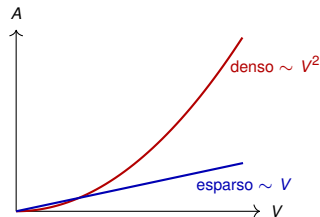
Um grafo com V vértices tem no máximo $V(V - 1)$ arcos — um pouco menos que V^2 .

- Um grafo é **completo** se todo par ordenado de vértices distintos é um arco: tem exatamente $V(V - 1)$ arcos.
- Um **torneio** tem, para cada par $v w$ de vértices distintos, exatamente um dentre $v - w$ e $w - v$: tem $\frac{1}{2} V(V - 1)$ arcos.

Denso: A é da mesma ordem que V^2 (digamos $V^2/2$, $V^2/10$, ...). O tamanho é proporcional a V^2 .

Esparso: A é da mesma ordem que V (digamos $10V$, $V/2$, ...). O tamanho é proporcional a V .

Estas definições se aplicam a *famílias infinitas* de grafos, não a um grafo individual.



A diferença decide qual estrutura de dados vale a pena.

O que aprendemos

- Leque de saída/entrada; graus de saída (*outdegree*) e de entrada (*indegree*).
- Fonte, sorvedouro e vértice isolado.
- Soma dos graus de saída = soma dos graus de entrada = A .
- No máximo $V(V - 1)$ arcos; completo tem $V(V - 1)$, torneio tem $\frac{1}{2} V(V - 1)$.
- Denso ($A \sim V^2$) versus esparso ($A \sim V$).

Grafos não-dirigidos e subgrafos

Problema

Amizade, estradas de mão dupla e cabos de rede são relações *simétricas*. Modelá-las com arcos dirigidos obriga a lembrar sempre de inserir os dois sentidos.

Solução

Tratar cada par de arcos antiparalelos como uma entidade nova — a **aresta**. O grafo continua sendo o mesmo objeto; muda apenas como falamos dele.

Definição

Um grafo é **não-dirigido** se cada um de seus arcos é antiparalelo a algum outro arco: para todo arco $v-w$, o grafo também tem o arco $w-v$.

Cada par de arcos antiparalelos é uma **aresta** (*edge*). Se E é o número de arestas e A o de arcos, então $E = A/2$.

Vocabulário

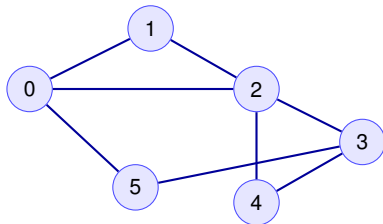
Uma aresta $v-w$ **liga** v e w , e **incide** em v e em w . O **leque** de v é o conjunto de arestas que incidem em v ; o **grau** de v é o número dessas arestas. A soma dos graus vale $2E$.

Um grafo não-dirigido

O conjunto de *arcos*

0-1 0-2 0-5 1-2 2-3 2-4 3-4 3-5 1-0 2-0 5-0 2-1 3-2 4-2 4-3 5-3

define um grafo não-dirigido, cujo conjunto de *arestas* é 0-1 0-2 0-5 1-2 2-3 2-4 3-4 3-5 ($A = 16$, $E = 8$).



A adjacência aqui é simétrica; o grau de 2 é 4 e o de 1 é 2.

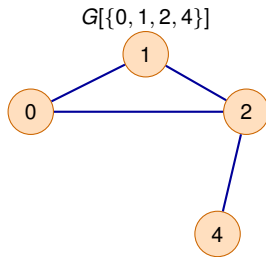
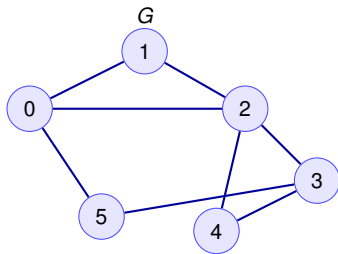
Definição

Um grafo H é **subgrafo** de um grafo G (notação $H \subseteq G$) se todo vértice de H é vértice de G e todo arco de H é arco de G .

- **gerador** (*spanning*): contém *todos* os vértices de G ;
- **induzido**: todo arco de G com ambas as pontas em H também é arco de H ;
- **próprio**: diferente de G (notação $H \subset G$).

Dado um conjunto X de vértices, $G[X]$ é o subgrafo induzido por X . Se a é um arco, $G - a$ é o subgrafo obtido removendo a .

Exemplo: subgrafo induzido



O subgrafo induzido por $X = \{0, 1, 2, 4\}$ mantém *todas* as arestas de G com as duas pontas em X — nem uma a mais, nem uma a menos.

O que aprendemos

- Não-dirigido: todo arco tem seu antiparalelo; cada par é uma **aresta**.
- $E = A/2$; soma dos graus $= 2E$.
- Subgrafo $H \subseteq G$; gerador, induzido, próprio.
- $G[X]$: subgrafo induzido por um conjunto X de vértices.
- Um subgrafo de um grafo não-dirigido pode *não* ser não-dirigido.

Matriz de adjacências

Problema

A definição matemática de grafo não diz *nada* sobre como guardá-lo na memória. Sem uma representação concreta não há algoritmo, não há programa em C, não há análise de consumo de tempo.

Solução

Duas representações clássicas. Antes delas, uma convenção:

```
#define vertex int
```

Os vértices são sempre $0\ 1\ 2\ \dots\ V - 1$.

Definição

A **matriz de adjacências** de um grafo é uma matriz booleana com linhas e colunas indexadas pelos vértices:

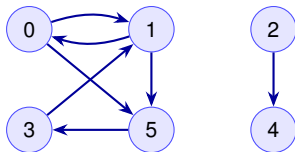
$$\text{adj}[v][w] = \begin{cases} 1 & \text{se } v-w \text{ é um arco,} \\ 0 & \text{caso contrário.} \end{cases}$$

Leitura da matriz

A *linha* v representa o leque de **saída** de v ; a *coluna* w representa o leque de **entrada** de w . Como não há laços, a diagonal é toda nula. Se o grafo é não-dirigido, a matriz é *simétrica*: $\text{adj}[v][w] \equiv \text{adj}[w][v]$.

A matriz do nosso exemplo

Arcos 0-1 0-5 1-0 1-5 2-4 3-1 5-3:



	0	1	2	3	4	5
0	0	1	0	0	0	1
1	1	0	0	0	0	1
2	0	0	0	0	1	0
3	0	1	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	1	0	0

A linha 4 é toda nula: 4 é um **sorvedouro**. A coluna 2 é toda nula: 2 é uma **fonte**.

```
/* REPRESENTACAO POR MATRIZ DE ADJACENCIAS:
   O campo adj e um ponteiro para a matriz de adjacencias do grafo.
   O campo V contem o numero de vertices e A o numero de arcos. */
struct graph {
    int V;
    int A;
    int **adj;
};

/* Um Graph e um ponteiro para um graph. */
typedef struct graph *Graph;
```

O grafo carrega consigo V e A : assim, perguntar quantos vértices ou quantos arcos o grafo tem custa tempo constante.

```
/* Constroi um grafo com vertices 0 1 .. V-1 e nenhum arco. */
Graph GRAPHinit( int V) {
    Graph G = malloc( sizeof *G);
    G->V = V;
    G->A = 0;
    G->adj = MATRIXint( V, V, 0);
    return G;
}

/* Aloca matriz com linhas 0..r-1 e colunas 0..c-1, tudo com valor val. */
static int **MATRIXint( int r, int c, int val) {
    int **m = malloc( r * sizeof (int *));
    for (vertex i = 0; i < r; ++i)
        m[i] = malloc( c * sizeof (int));
    for (vertex i = 0; i < r; ++i)
        for (vertex j = 0; j < c; ++j)
            m[i][j] = val;
    return m;
}
```

```
/* Insere um arco v-w no grafo G. Supoe v e w distintos, positivos
   e menores que G->V. Se o arco ja existe, nao faz nada. */
void GRAPHinsertArc( Graph G, vertex v, vertex w) {
    if (G->adj[v][w] == 0) {
        G->adj[v][w] = 1;
        G->A++;
    }
}

/* Remove do grafo G o arco v-w. Se o arco nao existe, nao faz nada. */
void GRAPHremoveArc( Graph G, vertex v, vertex w) {
    if (G->adj[v][w] == 1) {
        G->adj[v][w] = 0;
        G->A--;
    }
}
```

Ambas consomem tempo **constante**: um acesso à matriz decide tudo.

```
/* Imprime, para cada vertice v do grafo G, em uma linha,
   todos os vertices adjacentes a v. */
void GRAPHshow( Graph G) {
    for (vertex v = 0; v < G->V; ++v) {
        printf( "%2d:", v);
        for (vertex w = 0; w < G->V; ++w)
            if (G->adj[v][w] == 1)
                printf( " %2d", w);
        printf( "\n");
    }
}
```

Custo

Percorrer os vizinhos de *um* vértice custa tempo proporcional a V , mesmo que ele tenha um único vizinho:
GRAPHshow() custa V^2 .

O que aprendemos

- $\text{adj}[v][w] \in \{0, 1\}$; linha = leque de saída, coluna = leque de entrada.
- Diagonal nula (sem laços); matriz simétrica se o grafo é não-dirigido.
- Espaço proporcional a V^2 — *proporcional ao tamanho* apenas se o grafo é denso.
- Testar se $v-w$ é arco, inserir e remover: tempo constante.
- Percorrer os vizinhos de um vértice: tempo proporcional a V .

Listas de adjacência

Problema

Um grafo esparsos com $V = 10^6$ vértices e $A = 5 \cdot 10^6$ arcos exigiria 10^{12} células de matriz — quase todas iguais a zero. Impossível na prática.

Solução

Guardar, para cada vértice, apenas a lista dos vizinhos que ele realmente tem. O espaço passa a ser proporcional ao **tamanho** $V + A$ do grafo.

Definição

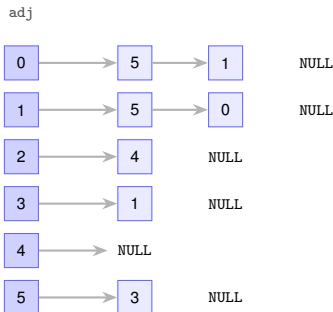
O **vetor de listas de adjacência** de um grafo tem uma lista encadeada associada a cada vértice. A lista associada a v contém todos os vizinhos de v ; portanto, ela representa o **leque de saída** de v .

Exemplo

Para os arcos 0-1 0-5 1-0 1-5 2-4 3-1 5-3:

0: 5 1 1: 5 0 2: 4 3: 1 4: 5: 3

A *ordem* dentro de cada lista depende da ordem de inserção — ela não faz parte da definição do grafo.



O vetor tem V posições; somando todos os nós das listas temos exatamente A nós. Daí o espaço proporcional a $V + A$.

```
/* Cada no da lista corresponde a um arco: contem um vizinho w de v
   e o endereco do no seguinte. Um link e um ponteiro para um node. */
typedef struct node *link;
struct node {
    vertex w;
    link next;
};

/* O campo adj e um ponteiro para o vetor de listas de adjacencia. */
struct graph {
    int V;
    int A;
    link *adj;
};
typedef struct graph *Graph;
```

```
/* Recebe um vertice w e o endereco next de um no e devolve o endereco
   a de um novo no tal que a->w == w e a->next == next. */
static link NEWnode( vertex w, link next) {
    link a = malloc( sizeof (struct node));
    a->w = w;
    a->next = next;
    return a;
}

/* Constroi um grafo com vertices 0 1 .. V-1 e nenhum arco. */
Graph GRAPHinit( int V) {
    Graph G = malloc( sizeof *G);
    G->V = V;
    G->A = 0;
    G->adj = malloc( V * sizeof (link));
    for (vertex v = 0; v < V; ++v)
        G->adj[v] = NULL;
    return G;
}
```

```
/* Insere um arco v-w no grafo G. Supoe v e w distintos, positivos
   e menores que G->V. Se o grafo ja tem o arco v-w, nao faz nada. */
void GRAPHinsertArc( Graph G, vertex v, vertex w) {
    for (link a = G->adj[v]; a != NULL; a = a->next)
        if (a->w == w) return;
    G->adj[v] = NEWnode( w, G->adj[v]);
    G->A++;
}
```

Custo

A função consome muito tempo (no pior caso, tempo proporcional ao *número de arcos*), pois percorre a lista para verificar se o arco a inserir já existe. O novo nó entra sempre no *início* da lista — por isso a ordem observada no exemplo é o inverso da ordem de inserção.

O que aprendemos

- Vetor de V listas encadeadas; a lista de v é o leque de saída de v .
- `struct node` guarda o vizinho w e o ponteiro `next`.
- Espaço proporcional a $V + A$, ou seja, ao **tamanho** do grafo.
- Percorrer os vizinhos de v custa tempo proporcional ao grau de saída de v .
- `GRAPHinsertArc()` é cara porque evita arcos repetidos.

Qual representação escolher?

	Matriz de adjacências	Listas de adjacência
Espaço	proporcional a V^2	proporcional a $V + A$
“ $v-w$ é arco?”	tempo constante	proporcional ao grau de saída de v
Percorrer vizinhos de v	proporcional a V	proporcional ao grau de saída de v
Inserir arco	tempo constante	proporcional ao número de arcos
Grafos densos	apropriada	sem vantagem
Grafos esparsos	desperdiça memória	econômica

Não existe representação universalmente melhor: a escolha depende da densidade do grafo e das operações que o *seu* algoritmo executa com mais frequência.

Arquivo de arcos

Linha 1: V . Linha 2: A . Depois, A linhas com dois inteiros em $0..V-1$.

```
7
6
0 1
0 5
1 0
1 5
2 4
3 1
```

Arquivo de adjacências

Linha 1: V . Depois, V linhas: a que começa com v lista todos os vizinhos de v .

```
7
0 1 6
1 0 6 3
2 4
3 1
4 2
5
6 0 1
```

Em ambos os casos a leitura se faz com `GRAPHinit()` seguido de chamadas a `GRAPHinsertArc()`.

Exercícios sugeridos (Feofiloff)

1. Escreva `GRAPHinddeg()` e `GRAPHoutdeg()` nas duas representações.
2. Calcule o vetor booleano `isSink[]` que identifica os sorvedouros.
3. Decida se dois vértices são adjacentes; quanto tempo isso consome em cada representação?
4. Escreva `GRAPHremoveArc()` para listas de adjacência.
5. Escreva funções que convertam uma representação na outra.
6. Escreva `UGRAPHinsertEdge()` e `UGRAPHdegrees()` para grafos não-dirigidos.

O tutorial em Jupyter que acompanha esta aula traz esqueletos em C para os itens 1, 4 e 5.

As cinco ideias centrais

- Grafo é um par (vértices, arcos); arco é par *ordenado* $v-w$.
- Graus resumem a estrutura local; $\sum \text{grau de saída} = A \leq V(V-1)$.
- Grafos não-dirigidos: pares antiparalelos vistos como arestas, $E = A/2$.
- Matriz de adjacências: espaço V^2 , adjacência em tempo constante.
- Listas de adjacência: espaço $V + A$, percurso de vizinhos proporcional ao grau.

Próxima aula: caminhos, ciclos e busca em profundidade (DFS) — os primeiros algoritmos que *usam* estas representações.



Jon Bentley.
Programming Pearls.
Addison-Wesley, 2 edition, 2000.



Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan.
Time bounds for selection.
Journal of Computer and System Sciences, 7(4):448–461, 1973.
Artigo original da mediana das medianas: seleção em $O(n)$ no pior caso.



J. Adrian Bondy and U. S. R. Murty.
Graph Theory, volume 244 of *Graduate Texts in Mathematics*.
Springer, 2008.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Paulo Feofiloff.

Algoritmos para grafos (em linguagem C).

Material didático online, DCC/IME-USP, 2020.

Capítulos *Grafos* e *Estruturas de dados para grafos*. Acesso em 01 set. 2026.



Paulo Feofiloff, Yoshiharu Kohayakawa, and Yoshiko Wakabayashi.

Uma introdução sucinta à teoria dos grafos.

Texto didático online, IME-USP, 2011.

Referência em português para as definições de teoria dos grafos. Acesso em 01 set. 2026.



Michael R. Garey and David S. Johnson.

Computers and Intractability: A Guide to the Theory of NP-Completeness.

W. H. Freeman, 1979.



David Gries.

The Science of Programming.

Springer-Verlag, 1981.



Michael Held and Richard M. Karp.

A dynamic programming approach to sequencing problems.

Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210, 1962.



C. A. R. Hoare.

Algorithm 64: Quicksort.

Communications of the ACM, 4(7):321, 1961.



C. A. R. Hoare.

Algorithm 65: FIND.

Communications of the ACM, 4(7):321–322, 1961.

O quickselect, publicado por Hoare no mesmo ano do quicksort.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Donald E. Knuth.

The Art of Computer Programming, Volume 3: Sorting and Searching.

Addison-Wesley, 2 edition, 1998.

Seção 5.3.3: seleção mínima em número de comparações.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



David R. Musser.

Introspective sorting and selection algorithms.

Software: Practice and Experience, 27(8):983–993, 1997.

O introselect: híbrido entre quickselect e mediana das medianas.



Robert Sedgewick.

Algorithms in C, Part 5: Graph Algorithms.

Addison-Wesley, 3 edition, 2002.

Fonte dos exemplos e exercícios do Capítulo 17 citados por Feofiloff.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.



Nivio Ziviani.

Projeto de Algoritmos com Implementações em Pascal e C.

Cengage Learning, 3 edition, 2011.

Capítulo 7: algoritmos em grafos.